

Reconstructed Workflow of the Thesis Workspace

Workspace Audit Report

24 April 2026

Abstract

This document summarizes the technical workflow implemented in the workspace by reading the project structure, scripts, and configuration files. It explains the sequence of modeling, simulation, processing, imaging, inversion, and uncertainty analysis steps, and it links these steps to concrete folders and Python entry points. In particular, the `Diffraction_Experiment` branch is framed as a *detection* campaign for subwavelength change visibility, while the remaining inversion and Bayesian branches are framed as *inference* pipelines for parameter recovery. The report is reconstructed from available files, so it represents the implemented pipeline rather than an exact historical execution log.

1 Workspace-Level View

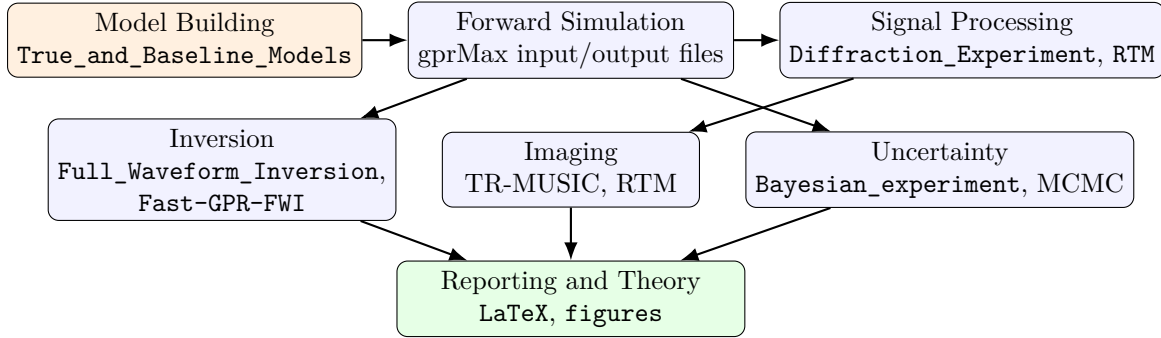


Figure 1: High-level architecture of the workspace workflow.

2 Step-by-Step Technical Process

Step 1: Define Physics and Resolution Constraints

The utility functions in `True_and_Baseline_Models/helper.py` encode key physical constraints:

- Central wavelength in a dielectric medium,
- Sub-wavelength fracture thickness threshold,
- Depth constraints and lateral resolution heuristics.

This indicates that the workflow starts from theoretical detectability limits before simulation setup.

Step 2: Build Baseline and True/Monitor Models

The folder `True_and_Baseline_Models` contains baseline and process models (for example, `baseline_model.in`, `process_model.in`), with helpers to modify fracture geometry and material properties. The script `update_aperture.py` exposes aperture sweeps via command-line arguments, which suggests controlled parametric studies on thin-bed/fracture geometry.

Step 3: Run Forward Simulations (gprMax)

Multiple subprojects generate and run gprMax files:

- `RTM/generate_and_run.py` creates a multi-shot acquisition over subwavelength scatterers,
- `Diffraction_Experiment/run_simulation_and_plot.py` triggers forward simulations and immediate plotting,
- `Diffraction_Experiment/run_multi_tx_trmusic.py` automates multi-transmitter data acquisition.

At this stage, raw `.out` files are produced for both target and homogeneous/baseline scenarios.

Step 4: Differential Data Construction

A recurring pattern in the code is subtraction of baseline/homogeneous responses from target responses to isolate weak scattered components:

$$\delta d(t, x_{tx}, x_{rx}) = d_{\text{target}}(t, x_{tx}, x_{rx}) - d_{\text{baseline}}(t, x_{tx}, x_{rx}).$$

This differential strategy appears in RTM and TR-MUSIC pipelines and is also consistent with the MCMC plan in `LaTeX/plan.tex`.

Step 5: Advanced Signal Processing and Feature Enhancement

The object-oriented workflow in `Diffraction_Experiment/processing_00.py` and `run_simulation_and_plot.py` applies:

- wiggle-trace diagnostics,
- auto/cross-correlation,
- CWT amplitude and phase imaging,
- phase derivatives (time and frequency),
- geometric shift-and-correlate with XWT.

This reveals a strong emphasis on extracting subtle diffraction signatures and phase behavior as a **detection** problem: the objective is to determine whether a subwavelength perturbation is observable above background and direct-wave clutter.

Step 6: Imaging with TR-MUSIC and RTM

Two imaging routes coexist:

- **TR-MUSIC branch:** `Diffraction_Experiment/trmusic_diffraction.py` and multi-Tx runners,
- **RTM branch:** `RTM/process_rtm.py` (frequency-domain reflection matrix backpropagation with FMC geometry).

Both branches transform processed multistatic data into spatial focusing maps of likely scatterer locations.

Step 6.5: Detection Branch vs Inference Branches

The project is split into two methodological goals:

- **Detection goal (Diffraction_Experiment):** establish whether subwavelength changes produce statistically and visually separable signatures in differential traces, CWT/XWT maps, and TR-MUSIC pseudo-spectra.
- **Inference goal (other folders):** estimate physical parameters (for example aperture, permittivity, conductivity, or velocity-related fields) using optimization or Bayesian posterior sampling.

In short, `Diffraction_Experiment` answers “*Can we see it?*” and the inversion/Bayesian branches answer “*What is it?*”.

Step 7: Full Waveform Inversion (FWI)

The folder `Full_Waveform_Inversion` contains a Devito-based inversion stack:

- forward/adjoint operator construction,
- objective and gradient evaluation,
- bounded L-BFGS-B optimization,
- data preparation scripts and plotting utilities.

The script `run_fwi.py` includes explicit Windows compatibility patches for Devito memory allocation and compilation paths, confirming active adaptation to the local OS environment.

Step 8: Accelerated Dual-Parameter FWI (GPU/PyTorch/CUDA)

The folder `Fast-GPR-FWI` introduces a second inversion track focused on speed and dual-parameter recovery (permittivity and conductivity), with CUDA-accelerated kernels integrated into PyTorch according to `Fast-GPR-FWI/README.md`.

Step 9: Bayesian Inference and Uncertainty Quantification

The script `Bayesian_experiment/Bayesian_inference.py` implements a Metropolis-Hastings loop:

- propose fracture parameter values,
- regenerate gprMax input,
- run simulation,
- compare with observed data through a Gaussian likelihood.

This yields posterior traces for model parameters and complements deterministic inversion with uncertainty estimates.

Step 10: Alternative Misfit Research Track

The folder `Quantum_Geometric_Misfits-main` contains scripts for L2 and geometric/quantum-inspired misfit experiments, including batch execution via `run_in_sequence.py`. This indicates method-comparison work beyond classical least-squares objectives.

3 Diffraction_Experiment as a Detection Campaign

The folder `Diffraction_Experiment` is the dedicated observability testbed for subwavelength change detection. Its internal structure maps naturally to a detection workflow:

- `forward.py`: build and run alternating and homogeneous forward models.
- `processing.py`, `processing_00.py`: construct differential traces and apply signal conditioning (correlation, wavelet analysis, phase diagnostics).
- `plotting.py`, `plotting_00.py`: produce diagnostic figures used to decide if change-related energy is resolvable.
- `run_simulation_and_plot.py`: end-to-end driver for simulation + detection analytics.
- `run_multi_tx_trmusic.py`, `run_multi_tx_trmusic_gapped.py`, `trmusic_diffraction.py`: multi-Tx acquisition and TR-MUSIC pseudo-spectrum imaging for location-level detectability.
- `horizontal_scattering_0p5lambda_snaps`, `horizontal_scattering_homogeneous_snaps`: forward-wavefield snapshots for qualitative comparison.
- `cwt_3d_traces`: per-trace wavelet images for trace-by-trace detectability checks.

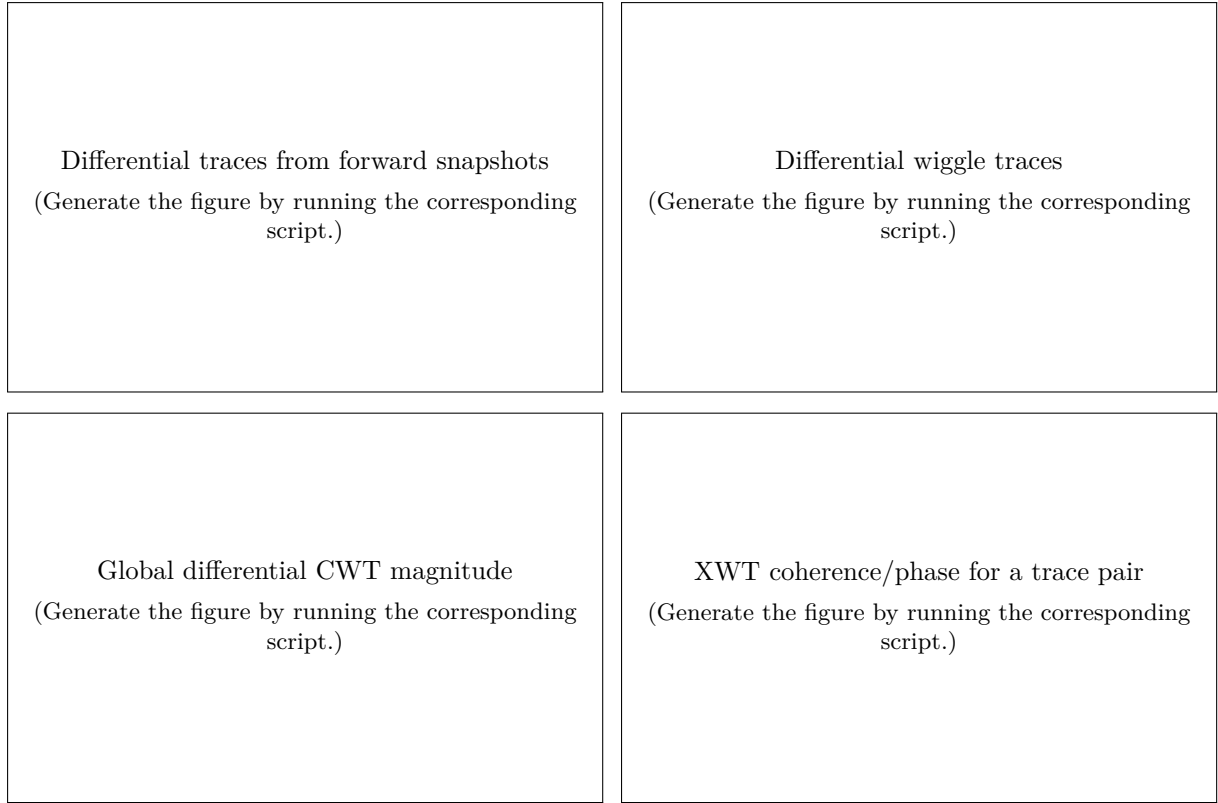


Figure 2: Detection-oriented diagnostics in `Diffraction_Experiment`: differential response isolation, time-frequency concentration, and inter-trace phase coherence.

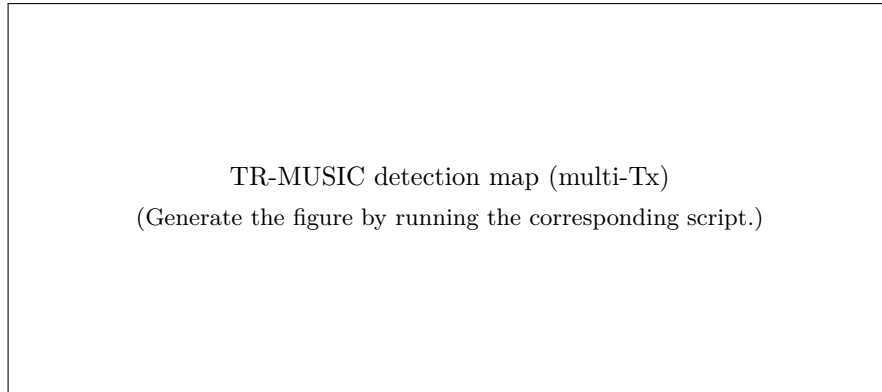


Figure 3: TR-MUSIC pseudo-spectrum in the diffraction branch, used as a detectability indicator for subwavelength perturbations.

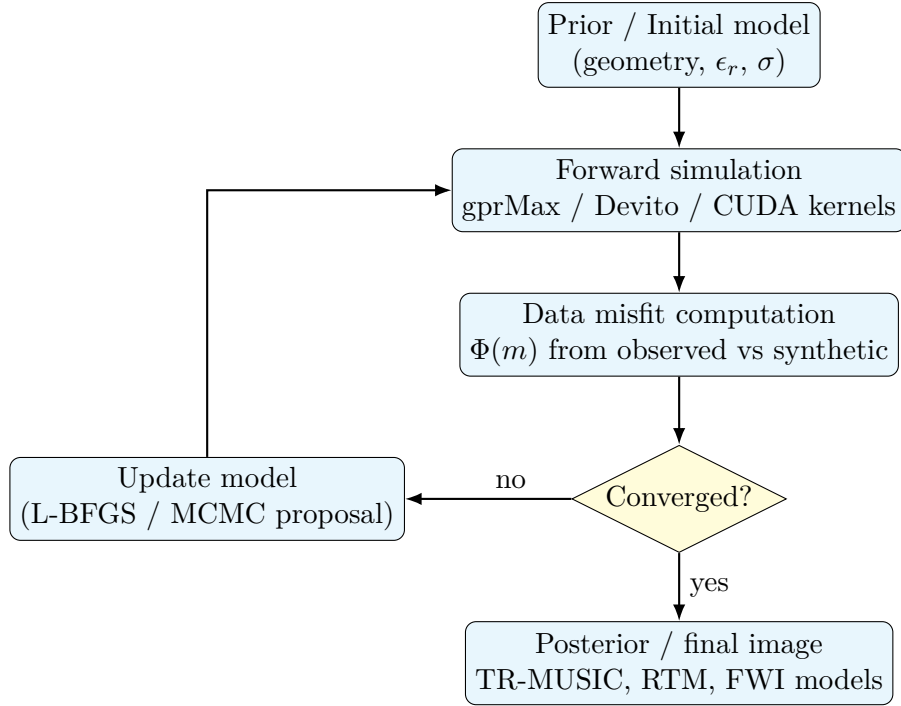


Figure 4: Unified optimization/inference loop implemented across deterministic and Bayesian pipelines.

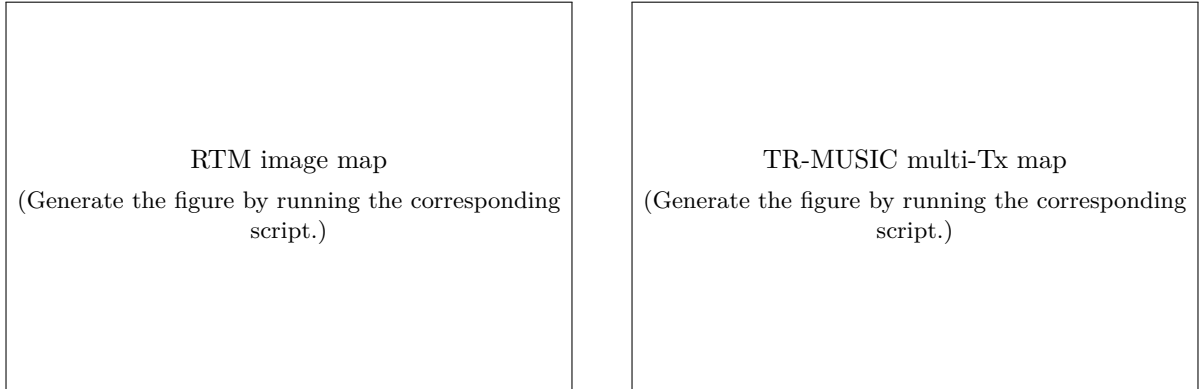


Figure 5: Auto-loaded outputs if available; placeholders are shown otherwise.

4 Core Research Loop

5 Representative Outputs

6 Execution Timeline (Reconstructed)

7 Directory-to-Task Mapping

8 Closing Notes

The workspace reflects a two-layer research strategy: a detection layer (centered on `Diffraction_Experiment`) to validate subwavelength observability, and an inference layer (FWI, accelerated FWI, Bayesian, and misfit-analysis branches) to estimate the underlying physical change once detectability is

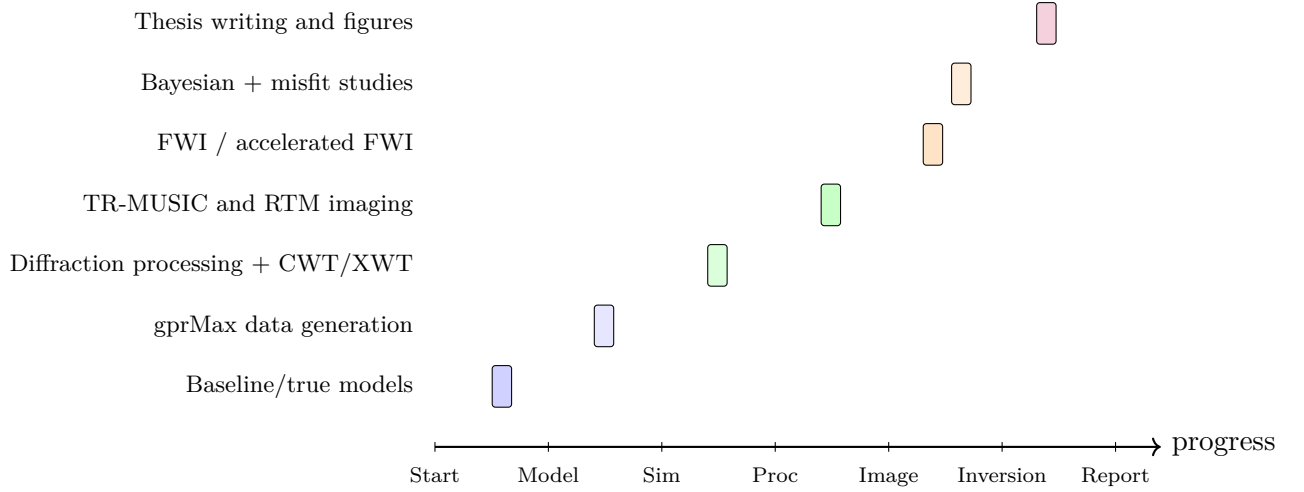


Figure 6: Reconstructed sequence of work packages across the repository.

Directory	Main role in the workflow
<code>True_and_Baseline_Models</code>	Geometry design, baseline/process model generation, aperture and material parameter control.
<code>Diffraction_Experiment</code>	Detection branch: differential signal analysis, CWT/XWT feature extraction, and TR-MUSIC-based observability tests for subwavelength changes (“can we detect it?”).
<code>RTM</code>	FMC survey generation and reflection-matrix-based RTM imaging.
<code>Full_Waveform_Inversion</code>	Devito forward/adjoint operators and bounded L-BFGS inversion.
<code>Fast-GPR-FWI</code>	CUDA + PyTorch accelerated dual-parameter FWI experiments.
<code>Bayesian_experiment</code>	Metropolis-Hastings parameter inference and posterior trace generation (“what is the change?”).
<code>Quantum_Geometric_Misfits-main</code>	Alternative objective-function experiments (L2 and geometric tracks).
<code>LaTeX</code>	Theory notes, planning, and manuscript/report material.

Table 1: How each major folder contributes to the end-to-end research process.

established. A practical next step is to append run metadata (date, command, git commit hash, runtime, key parameters) to each experiment so future reports can be generated as exact execution logs.